

APUNTE 1

Permisos y Controles Básicos en UNIX/Linux

En los sistemas UNIX/Linux, el control de acceso a recursos es uno de los mecanismos de seguridad más fundamentales. Todo en UNIX es un archivo: los documentos, los dispositivos, los directorios, incluso los procesos. Por eso, entender cómo funciona el sistema de permisos es entender cómo el sistema operativo decide quién puede hacer qué.

Este apunte cubre los conceptos base del modelo de seguridad UNIX: cómo se identifican los usuarios y grupos, qué rol tiene el superusuario, cómo se leen e interpretan los permisos de archivos y directorios, y cómo se modifican con las herramientas del sistema.

1. Identificación de Usuarios y Grupos

Para que el sistema operativo pueda controlar quién accede a qué, primero necesita identificar a los usuarios de forma única. UNIX lo hace a través de identificadores numéricos.

1.1 UID y GID

Cada usuario del sistema tiene asignado un UID (User Identifier) y cada grupo tiene un GID (Group Identifier). Ambos son números enteros de 16 bits (rango 0–65535) que el kernel usa internamente para tomar decisiones de control de acceso.

Cuando un usuario ejecuta un proceso, ese proceso hereda el UID y GID del usuario. El kernel compara esos identificadores con los permisos del archivo o recurso al que se intenta acceder y decide si la operación está permitida.

Rango de UID	Tipo	Descripción
0	Superusuario (root)	Privilegios totales. Puede hacer cualquier cosa en el sistema.
1 – 99	Cuentas del sistema	Reservadas para servicios del SO (daemon, bin, sys, etc.).
100 – 999	Cuentas de sistema dinámicas	Creadas por paquetes instalados (www-data, postgres, etc.).
1000 en adelante	Usuarios normales	Cuentas de usuarios reales del sistema.

```
# Ver el UID y GID del usuario actual
id
# Salida: uid=1001(alice) gid=1001(alice) groups=1001(alice),27(sudo)

# Ver todos los usuarios del sistema
cat /etc/passwd

# Formato de cada línea en /etc/passwd:
```

```
# usuario:x:UID:GID:descripción:home:shell
alice:x:1001:1001:Alice García:/home/alice:/bin/bash

# La 'x' en el segundo campo indica que la contraseña está en /etc/shadow
```

1.2 Principals, Sujetos y Objetos

El modelo de seguridad de UNIX se puede describir en términos de tres conceptos que aparecen en toda la literatura de sistemas operativos:

- **Principal:** la entidad que tiene identidad en el sistema. Generalmente es un usuario identificado por su UID, aunque también puede ser un grupo.
- **Sujeto:** el proceso que actúa en nombre de un principal. Cuando Alice abre un editor de texto, el proceso del editor es el sujeto. Hereda los permisos de Alice.
- **Objeto:** el recurso sobre el cual se realiza una acción. Puede ser un archivo, un directorio, un dispositivo, un socket, etc.

Ejemplo: Alice (principal) abre el archivo documento.txt (objeto) con el editor vi (sujeto). El kernel verifica si el UID de Alice tiene permiso de lectura sobre documento.txt antes de permitir la operación.

Esta distinción es importante porque los permisos en UNIX se aplican al proceso, no directamente al usuario. Si Alice ejecuta un programa que a su vez intenta acceder a un archivo, el acceso se evalúa con los permisos del proceso, que normalmente son los de Alice — salvo casos especiales como SUID, que veremos en el Apunte 3.

2. El Usuario Root

El usuario con UID 0 es el superusuario, conocido como root. Es la cuenta con privilegios absolutos en el sistema: puede leer cualquier archivo independientemente de sus permisos, modificar la configuración del sistema, montar y desmontar sistemas de archivos, cambiar el propietario de cualquier archivo, y prácticamente cualquier otra operación.

2.1 Por qué root es peligroso

Precisamente porque no tiene restricciones, trabajar directamente como root es arriesgado. Un error tipográfico, un script mal escrito o un programa malicioso ejecutado como root puede destruir el sistema completo sin posibilidad de recuperación.

```
# Ejemplo clásico de accidente como root:
rm -rf / --no-preserve-root
# Esto elimina TODOS los archivos del sistema. Sin confirmación.
# Como usuario normal, fallaría al intentar borrar archivos del sistema.
# Como root, lo hace sin problemas.
```

Por ejemplo, si un archivo con SUID está configurado con permisos 777 y pertenece a root:

```
-rwsrwxrwx 1 root root /usr/bin/script_peligroso
```

Cualquier usuario podría modificar el script e insertar comandos maliciosos que se ejecutarían con privilegios de root la próxima vez que alguien ejecute el archivo. Este es un claro ejemplo de cómo los permisos incorrectos pueden llevar a vulnerabilidades críticas. Como recomendación siempre usar los permisos mínimos necesarios para el funcionamiento correcto

Nunca trabajar como root de forma permanente. Usar root solo para las tareas que lo requieran y volver inmediatamente al usuario normal.

2.2 sudo: la alternativa segura

El comando sudo (superuser do) permite a usuarios autorizados ejecutar comandos específicos con privilegios de root, sin necesidad de conocer la contraseña de root ni iniciar sesión como root. Cada uso de sudo queda registrado en los logs del sistema, lo que facilita la auditoría.

```
# Ejecutar un comando con privilegios de root
sudo cat /etc/shadow

# Ver quién tiene permisos sudo en el sistema
sudo cat /etc/sudoers

# Agregar un usuario al grupo sudo (en Debian/Ubuntu)
sudo usermod -aG sudo alice

# Los logs de sudo se guardan en:
cat /var/log/auth.log | grep sudo
```

sudo vs su: el comando su (switch user) permite cambiar completamente de usuario, incluyendo a root. La diferencia con sudo es que su requiere la contraseña del usuario destino y no registra los comandos ejecutados después del cambio. sudo es más seguro y auditable.

2.3 Capabilidades del kernel (capabilities)

En kernels modernos de Linux, los privilegios de root están divididos en unidades más pequeñas llamadas capabilities. En vez de dar acceso total, se puede otorgar a un proceso solo la capability específica que necesita. Algunos ejemplos:

Capability	Permite
CAP_NET_BIND_SERVICE	Abrir puertos menores a 1024 (HTTP, SSH, etc.)
CAP_CHOWN	Cambiar el propietario de cualquier archivo
CAP_SYS_ADMIN	Operaciones de administración del sistema
CAP_DAC_OVERRIDE	Ignorar permisos de archivos (DAC)
CAP_KILL	Enviar señales a cualquier proceso

```
# Ver las capabilities de un proceso
cat /proc/$(pidof sshd)/status | grep Cap

# Ver las capabilities de un binario
getcap /usr/bin/ping
```

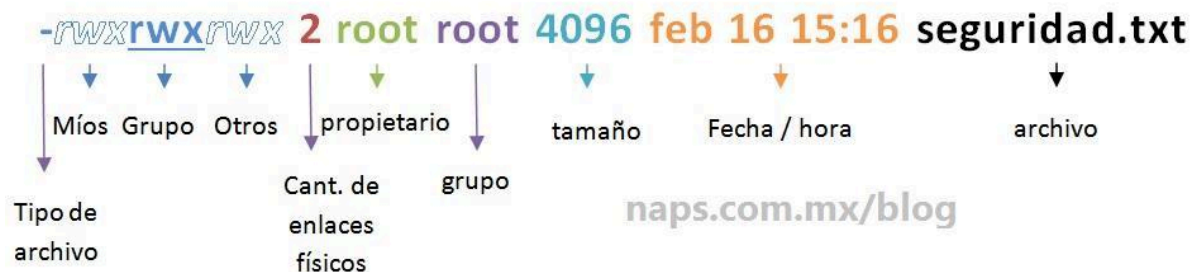
Las capabilities permiten el principio de mínimo privilegio: dar a cada proceso solo los permisos que necesita, no más. Esto reduce el daño potencial si ese proceso es comprometido.

3. El Modelo de Permisos DAC en UNIX

UNIX implementa un modelo de control de acceso llamado DAC (Discretionary Access Control — Control de Acceso Discrecional). La característica central de DAC es que el propietario de un recurso decide quién puede acceder a él. Es el modelo más flexible, pero también el más propenso a errores humanos.

En este modelo, cada archivo y directorio tiene asociados tres elementos de control:

- **Usuario propietario (owner):** el usuario que creó el archivo o al que se le asignó la propiedad. Se identifica por su UID.
- **Grupo propietario (group):** el grupo asociado al archivo. Se identifica por su GID.
- **Otros (others):** cualquier usuario que no sea el propietario ni pertenezca al grupo.



Para cada uno de estos tres, se definen tres tipos de permisos:

Símbolo	Nombre	En archivos	En directorios
r	Lectura	Ver el contenido del archivo	Listar el contenido del directorio (ls)
w	Escritura	Modificar o eliminar el archivo	Crear, renombrar o eliminar archivos dentro del directorio
x	Ejecución	Ejecutar el archivo como programa	Ingresa al directorio y acceder a su contenido (cd)
-	Sin permiso	La acción está denegada	La acción está denegada

El permiso de ejecución (x) en directorios es frecuentemente mal entendido. Sin él, no podés hacer `cd` al directorio ni acceder a los archivos dentro, aunque tengas permiso de lectura. El permiso `r` en directorios solo permite listar con `ls`, pero no acceder al contenido.

3.1 Cómo leer los permisos de un archivo

El comando `ls -l` muestra los permisos en formato simbólico. Veamos cómo interpretar la salida:

```
ls -l script.sh
-rwxr-xr-- 1 alice developers 4096 Jun 10 14:30 script.sh

# Desglose carácter por carácter:
# Posición 1:      - → archivo regular (d=directorio, l=enlace simbólico)
# Posiciones 2-4: rwx → permisos del propietario (alice): leer, escribir, ejecutar
# Posiciones 5-7: r-x → permisos del grupo (developers): leer, ejecutar (sin escribir)
# Posiciones 8-10:r-- → permisos de otros: solo leer

# Número '1'      → cantidad de enlaces duros
# alice           → usuario propietario
# developers      → grupo propietario
# 4096            → tamaño en bytes
# Jun 10 14:30    → fecha de última modificación
# script.sh       → nombre del archivo
```

Ejemplo: Con los permisos `rwxr-xr--`: Alice puede leer, escribir y ejecutar el script. Cualquier miembro del grupo `developers` puede leerlo y ejecutarlo, pero no modificarlo. Cualquier otro usuario solo puede leerlo.

3.2 Representación numérica (octal)

Cada permiso tiene un valor numérico. Los tres permisos de cada categoría se suman para obtener un dígito octal. Tres categorías producen tres dígitos:

Valor	Permiso	Símbolo	Combinaciones comunes
4	Lectura	r--	
2	Escritura	-w-	
1	Ejecución	--x	
6 (4+2)	Lectura + escritura	rw-	Archivos de datos (propietario)
5 (4+1)	Lectura + ejecución	r-x	Scripts (grupo/otros)
7 (4+2+1)	Todos los permisos	rwx	Propietario de scripts ejecutables
0	Sin permisos	---	Archivos completamente restringidos

```
# Ejemplos de permisos en notación octal:

chmod 644 archivo.txt    # rw-r--r-- (propietario lee/escrive, resto solo lee)
chmod 755 script.sh      # rwxr-xr-x (propietario todo, resto lee/ejecuta)
chmod 700 privado.txt    # rwx----- (solo el propietario tiene acceso)
chmod 640 config.conf    # rw-r----- (propietario lee/escrive, grupo solo lee)

# Equivalencia entre notación simbólica y octal:
# rwxr-xr-- = 7 5 4 = chmod 754
```

4. chmod, Cambiar Permisos

El comando chmod (change mode) permite modificar los permisos de archivos y directorios. Solo el propietario del archivo o root puede ejecutarlo.

4.1 Método simbólico

El método simbólico especifica a quién afecta el cambio, qué operación realizar y qué permiso modificar:

¿A quién?	¿Operación?	¿Qué permiso?	Descripción
u (user/owner)	+ (agregar)	r (lectura)	Modifica permisos del propietario
g (group)	- (quitar)	w (escritura)	Modifica permisos del grupo
o (others)	= (establecer exacto)	x (ejecución)	Modifica permisos de otros
a (all)			Modifica todos a la vez

```
# Agregar permiso de ejecución al propietario
chmod u+x script.sh

# Quitar permiso de escritura al grupo
chmod g-w archivo.txt

# Establecer solo lectura para otros
chmod o=r archivo.txt

# Agregar ejecución a todos (propietario, grupo y otros)
chmod a+x script.sh

# Combinaciones en un solo comando:
chmod u=rwx,g=rx,o=r script.sh    # Equivalente a chmod 754 script.sh

# Aplicar permisos recursivamente a un directorio y todo su contenido
chmod -R 755 /var/www/html
```

4.2 Método numérico (octal)

```
# Sintaxis: chmod [permisos_en_octal] archivo

chmod 644 informe.pdf      # rw-r--r-- (documento normal)
chmod 755 servidor.sh     # rwxr-xr-x (script ejecutable)
chmod 700 claves.txt       # rwx----- (solo el propietario)
chmod 600 id_rsa           # rw----- (clave SSH privada, estándar)
chmod 750 /home/alice/     # rwxr-x--- (directorio personal semi-privado)

# Para calcular el octal mentalmente:
# rwxr-xr-- → 7(rwx) 5(r-x) 4(r--) → 754
```

Permisos recomendados según el tipo de archivo: archivos de datos → 644, scripts ejecutables → 755, archivos de configuración con credenciales → 600 o 640, directorios públicos → 755, directorios privados → 700 o 750.

5. chown y chgrp — Cambiar Propietario y Grupo

Además de los permisos, cada archivo tiene un propietario y un grupo asignados. `chown` (change owner) permite cambiar ambos, y `chgrp` (change group) permite cambiar solo el grupo.

Solo root puede cambiar el propietario de un archivo. El propietario actual puede cambiar el grupo del archivo, pero solo a un grupo al que pertenezca.

```
# Cambiar solo el propietario del archivo
sudo chown alice archivo.txt

# Cambiar propietario y grupo al mismo tiempo
sudo chown alice:developers archivo.txt

# Cambiar solo el grupo (dos formas equivalentes)
sudo chgrp developers archivo.txt
sudo chown :developers archivo.txt

# Aplicar cambio recursivamente a un directorio
sudo chown -R alice:developers /var/www/html

# Verificar el resultado
ls -l archivo.txt
# -rw-r--r-- 1 alice developers 1024 Jun 10 14:30 archivo.txt
```

Ejemplo: Un administrador acaba de crear un directorio `/proyectos/backend` para el equipo de desarrollo. Ejecuta: `sudo chown root:backend /proyectos/backend` y `sudo chmod 770 /proyectos/backend`. Resultado: root es el propietario, el grupo backend tiene acceso completo, y el resto del sistema no puede acceder.

6. umask, Permisos por Defecto

Cuando se crea un archivo o directorio, el sistema le asigna permisos automáticamente. ¿De dónde vienen esos permisos predeterminados? De la combinación entre los permisos máximos del sistema y la umask del usuario.

6.1 Cómo funciona la umask

El sistema de permisos en UNIX/Linux sigue una lógica fundamental de seguridad que diferencia entre archivos y directorios en su configuración por defecto:

- **Archivos:** permisos base 666 (rw-rw-rw-). Los archivos no tienen ejecución por defecto por razones de seguridad.
- **Directorios:** permisos base 777 (rwxrwxrwx). Los directorios necesitan ejecución para poder ingresar a ellos.

La umask es una máscara que se resta a esos permisos base. Con la umask más común (022), los permisos quedan así:

Tipo	Base	umask 022	Resultado
Archivo nuevo	666 (rw-rw-rw-)	- 022	644 (rw-r--r--)
Directorio nuevo	777 (rwxrwxrwx)	- 022	755 (rwxr-xr-x)

```
# Ver la umask actual
umask
# Salida: 0022

# Ver la umask en formato simbólico
umask -S
# Salida: u=rwx,g=rx,o=rx

# Cambiar la umask temporalmente (solo para la sesión actual)
umask 0027
# Ahora los archivos nuevos tendrán permisos 640 y los directorios 750

# Verificar creando un archivo de prueba
touch test.txt && ls -l test.txt
# -rw-r----- 1 alice alice 0 Jun 10 14:30 test.txt
```

6.2 Configuración permanente de la umask

Para que la umask persista entre sesiones, se debe configurar en el archivo de inicio del shell del usuario:

```
# Para Bash, agregar en ~/.bashrc o ~/.bash_profile:
echo 'umask 0027' >> ~/.bashrc

# Para configurar la umask del sistema completo (todos los usuarios):
```



```
# Editar /etc/login.defs o /etc/profile

# umask 0022 → estándar para servidores compartidos
# umask 0027 → más restrictivo: grupo puede leer, otros sin acceso
# umask 0077 → máxima privacidad: solo el propietario tiene acceso
```

6.3 ¿Por qué los archivos no deberían tener permisos 777?

Un archivo con permisos 777 (rwxrwxrwx) puede ser leído, modificado y ejecutado por cualquier usuario del sistema. Esto representa una vulnerabilidad crítica por tres razones:

- Cualquier usuario puede modificar el contenido e insertar código malicioso.
- Cualquier usuario puede ejecutarlo, potencialmente con consecuencias graves.
- En sistemas multiusuario, facilita ataques de escalamiento de privilegios si el archivo pertenece a un usuario privilegiado.

```
# Ejemplo peligroso: archivo SUID con permisos 777
-rwsrwxrwx 1 root root /usr/local/bin/script_admin

# Cualquier usuario puede modificar este script.
# La próxima vez que alguien lo ejecute, correrá con privilegios de root
# pero con el contenido que el atacante haya insertado.

# Esto es una vulnerabilidad crítica de escalamiento de privilegios.
```

Nunca asignar permisos 777 a archivos, especialmente si pertenecen a root o tienen SUID activo. Los permisos de escritura para otros (o=w) casi nunca son necesarios y siempre representan un riesgo.

7. Gestión de Usuarios y Grupos

La administración de usuarios y grupos es una tarea cotidiana en cualquier sistema multiusuario. Conocer los comandos y los archivos involucrados es fundamental para cualquier administrador de sistemas.

7.1 Comandos de gestión de usuarios

```
# Crear un nuevo usuario (forma interactiva, recomendada en Debian/Ubuntu)
sudo adduser alice

# Crear un usuario sin directorio home (para cuentas de servicio)
sudo useradd --no-create-home --shell /usr/sbin/nologin webservice

# Eliminar un usuario (sin eliminar su directorio home)
sudo userdel alice

# Eliminar un usuario y su directorio home
```

```
sudo userdel -r alice

# Modificar un usuario existente
sudo usermod -s /bin/zsh alice      # Cambiar shell
sudo usermod -aG sudo alice         # Agregar al grupo sudo
sudo usermod -L alice               # Bloquear cuenta
sudo usermod -U alice               # Desbloquear cuenta

# Cambiar contraseña de un usuario
sudo passwd alice
```

7.2 Comandos de gestión de grupos

```
# Crear un grupo nuevo
sudo groupadd developers

# Agregar un usuario a un grupo (sin sacarlo de otros grupos)
sudo usermod -aG developers alice

# Ver los grupos a los que pertenece un usuario
groups alice
# Salida: alice : alice developers sudo

# Ver todos los grupos del sistema
cat /etc/group

# Eliminar un grupo
sudo groupdel developers
```

El flag `-a` en `usermod -aG` es fundamental. Sin él, el usuario es removido de todos sus grupos actuales y solo queda en el nuevo. Con `-a` (append), se agrega al grupo sin tocar los demás.

7.3 Archivos del sistema relacionados

Archivo	Contenido
/etc/passwd	Información básica de usuarios: nombre, UID, GID, home, shell.
/etc/shadow	Hashes de contraseñas y políticas de expiración. Solo root.
/etc/group	Lista de grupos y sus miembros.
/etc/gshadow	Contraseñas de grupos y administradores de grupos.
/etc/sudoers	Define quién puede usar sudo y con qué restricciones.
/etc/security/opasswd	Historial de contraseñas anteriores (usado por PAM).

8. Buenas Prácticas de Permisos

Aplicar correctamente los permisos es tan importante como tenerlos disponibles. A continuación se presentan las recomendaciones más relevantes:

8.1 Principio de mínimo privilegio

Cada usuario, proceso o servicio debe tener únicamente los permisos necesarios para hacer su trabajo, y nada más. Si un servicio web solo necesita leer archivos estáticos, no debe tener permisos de escritura ni ejecución en el directorio raíz web.

8.2 Permisos recomendados por tipo de archivo

Tipo de archivo	Permisos	Justificación
Documentos y datos	644	Propietario escribe, todos leen
Scripts ejecutables	755	Propietario modifica, todos ejecutan
Archivos de configuración	600 o 640	Proteger credenciales y parámetros sensibles
Clave privada SSH	600	Solo el propietario puede leerla (SSH lo exige)
Directorios públicos	755	Acceso estándar para todos
Directorios privados	700 o 750	Acceso restringido al propietario o grupo
Directorios compartidos	770 o 775	Acceso completo al grupo, otros excluidos

8.3 Auditoría de permisos

Es recomendable revisar periódicamente los permisos del sistema para detectar configuraciones peligrosas:

```
# Buscar archivos con permisos 777 (peligroso)
find / -type f -perm 777 2>/dev/null

# Buscar archivos con bit SUID activo
find / -type f -perm -4000 2>/dev/null

# Buscar archivos sin propietario (pueden ser residuos de usuarios eliminados)
find / -nouser -o -nogroup 2>/dev/null

# Ver todos los archivos en un directorio con sus permisos
ls -la /etc/ | head -20
```

Referencias Bibliográficas

- Silberschatz, A. — Operating System Concepts, Cap. 14 (Protection)
- APUE (Advanced Programming in the UNIX Environment), Cap. 6
- OSTEP — Cap. 37 (Access Control)